

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE (SRS)

SISTEMA DE MARKETPLACE LOCAL — COMMERCITY MOBILE v2.0

Documento: SRS-CCM-2026-V2

Estándar de Referencia: IEEE Std 830-1998 / ISO/IEC/IEEE 29148

Proyecto: CommerCity Mobile (Capacitor / Android & iOS)

Organización: THE BLACK CODE JSB

Sprint: Cierre Sprint 2 (Fase de Especificación e Integración API)

Ambiente Backend: `http://localhost:3000` (Node.js / Express REST API)

Base de Datos Central: MySQL `commercity_v2` (149.130.178.228:3306)

Responsables de Coordinación: Yepes (Casos de Uso) / Equipo Móvil & Backend

1. INTRODUCCIÓN

1.1 Propósito

El presente documento define la Especificación de Requerimientos de Software (SRS) para el aplicativo móvil **CommerCity Mobile v2.0**. Establece los requerimientos funcionales, requerimientos no funcionales, reglas de negocio, modelos de seguridad y la arquitectura de integración con la API REST central (`http://localhost:3000`) y la base de datos relacional MySQL (`commercity_v2`), reemplazando el esquema de datos locales de la fase de prototipado.

1.2 Alcance del Sistema

CommerCity es un marketplace móvil diseñado para dinamizar el comercio local mediante la conexión directa entre compradores y vendedores. El sistema móvil permite:

- Exploración de catálogos por categorías, búsqueda y visualización de productos en tiempo real.
- Gestión de carrito de compras persistente con expiración automática.
- Procesamiento de pedidos y checkout con desglose fiscal y comisión calculados por el backend.
- Panel de administración de tienda e inventario para vendedores.
- Módulo de mensajería instantánea y chat entre comprador y comerciante.
- Consola administrativa y moderación de contenidos/usuarios para administradores de plataforma.
- Sistema de notificaciones operacionales y de envíos.

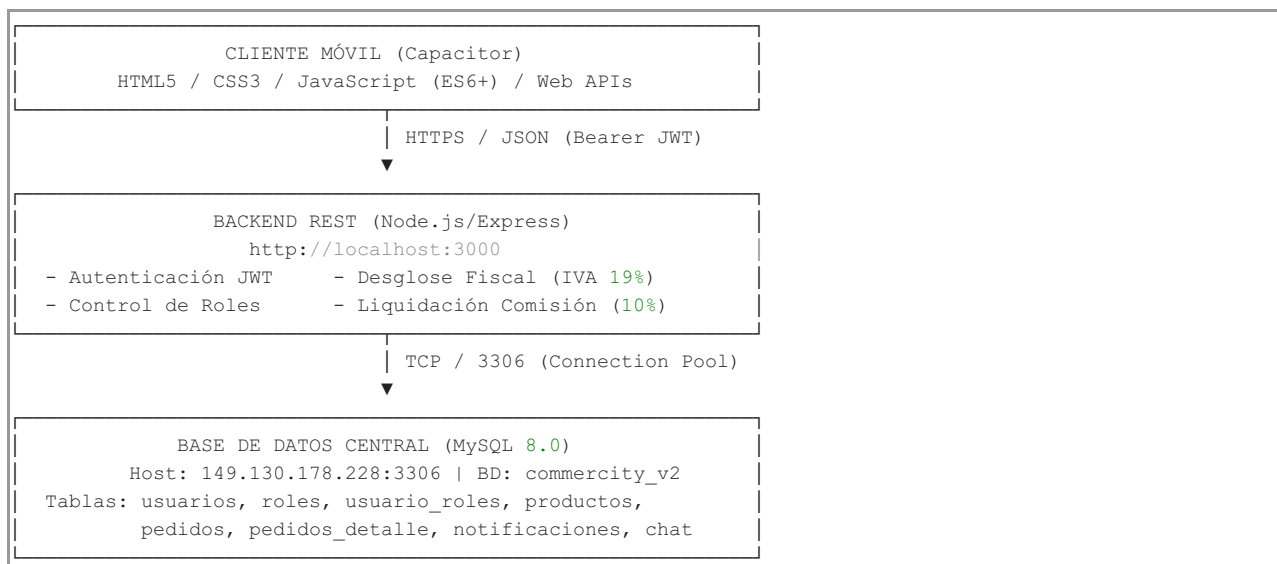
1.3 Definiciones, Acrónimos y Abreviaturas

- **SRS:** Software Requirements Specification (Especificación de Requerimientos de Software).
 - **JWT:** JSON Web Token (Estándar RFC 7519 para transmisión segura de identidad).
 - **REST:** Representational State Transfer.
 - **IVA:** Impuesto al Valor Agregado (19% nacional colombiano).
 - **Capacitor:** Runtime multiplataforma para empaquetado nativo web en Android e iOS.
 - **DBMS:** Database Management System (MySQL 8.0+).
-

2. DESCRIPCIÓN GENERAL

2.1 Perspectiva del Producto

CommerCity Mobile funciona como cliente desacoplado (frontend híbrido nativo) que consume servicios web expuestos por el backend central. Toda persistencia crítica, validación de stock, cálculo de comisiones e impuestos es responsabilidad exclusiva del servidor.



2.2 Perfiles de Usuario y Roles (RBAC)

La asignación de permisos se gestiona en la tabla `usuario_roles` de la base de datos `commercium_v2`:

Rol	Identificador BD	Descripción y Permisos
Comprador	COMPRADOR	Navegación de catálogo, carrito, compra de productos, historial de pedidos, calificaciones y chat con vendedores.
Vendedor	VENDEDOR	Capacidades de comprador + panel de tienda, publicación y edición de productos, control de stock, gestión de envíos y configuración bancaria.
Administrador	ADMINISTRADOR	Acceso a métricas globales de plataforma, moderación de reportes, suspensión/reactivación de usuarios y productos.

3. REQUERIMIENTOS FUNCIONALES (RF)

3.1 Módulo de Autenticación y Cuentas

- **RF01 - Inicio de Sesión Real (JWT):** El sistema autenticará al usuario contra `POST /api/usuarios/login`, validando credenciales y retomando un token JWT firmado junto con el array de roles (`usuario_roles`). Se elimina la validación mock de `email.includes('vendedor')` y el login hardcodeado de administrador.
- **RF02 - Registro de Usuario:** El sistema permitirá el registro de nuevos usuarios mediante `POST /api/usuarios/registro`, asignando rol base `COMPRADOR` y opcionalmente `VENDEDOR`.
- **RF03 - Recuperación de Contraseña:** El sistema permitirá solicitar enlace de restablecimiento mediante `POST /api/usuarios/recuperar`.
- **RF04 - Cierre de Sesión Seguro:** Al cerrar sesión, el cliente eliminará el token de autenticación y los datos de perfil locales, redirigiendo a la pantalla de login.

3.2 Módulo de Catálogo y Búsqueda

- **RF10 - Consulta de Catálogo:** El sistema obtendrá los productos disponibles mediante `GET /api/productos`, soportando filtros por categoría, búsqueda por texto y paginación.
- **RF11 - Detalle de Producto:** Al seleccionar un ítem, el sistema consultará `GET /api/productos/:id` para mostrar especificaciones, stock en tiempo real, descuentos aplicables y datos del vendedor.
- **RF22 - Feed Personalizado:** La vista de perfil dispondrá de una pestaña de recomendaciones sincronizada con la actividad del usuario.
- **RF23 - Mis Productos (Vendedor):** La vista de perfil listará los productos pertenecientes exclusivamente al comerciante autenticado.

3.3 Módulo de Carrito de Compras

- **RF30 - Persistencia en Servidor:** El carrito de compras se sincronizará con el backend vía `GET /api/carrito` y `POST /api/carrito`, asociándolo al identificador único del usuario autenticado.
- **RF31 - Control de Cantidades y Stock:** Al modificar la cantidad de un ítem (`PATCH /api/carrito/:id`), el backend validará la disponibilidad en inventario antes de autorizar el cambio.
- **RF32 - Eliminación de Ítems:** El sistema permitirá remover productos del carrito individualmente (`DELETE /api/carrito/:id`).
- **RF109 - Caducidad de Carrito Inactivo:** Los carritos sin actividad durante 7 días (604.800.000 ms) serán purgados.

automáticamente por el sistema.

3.4 Módulo de Pedidos, Checkout y Pagos

- **RF38 - Validación de Dirección:** El sistema exigirá que el comprador tenga registrada una dirección de entrega antes de habilitar el proceso de pago en la pasarela.
- **RF40 - Resumen Financiero Certificado:** Al abrir la pasarela, la app solicitará `GET /api/pedidos/resumen`. Todos los montos se computarán del lado del servidor:
 - **Subtotal:** Sumatoria de productos según cantidades y descuentos.
 - **IVA (19%):** Impuesto legal calculado por el servidor (RF134).
 - **Comisión de Plataforma (10%):** Tarifa de intermediación calculada por el backend sobre la venta para dispersión al vendedor.
 - **Total Definitivo:** Valor exacto a procesar. La app móvil **no realiza cálculos aritméticos de impuestos**.
- **RF41 - Confirmación de Transacción:** El sistema procesará el pago mediante `POST /api/pedidos/confirmar-pago`, ejecutando una transacción atómica que descuenta stock, crea el pedido, vacía el carrito y retoma el comprobante.
- **RF42 - Historial y Seguimiento:** El comprador podrá consultar sus compras en `GET /api/historial/compras` con filtrado por estado (Pendiente, En Camino, Entregado, Cancelado).
- **RF43 - Cancelación de Pedido:** El comprador podrá solicitar cancelación de pedidos en estado pendiente vía `POST /api/historial/cancelar/:id`.

3.5 Módulo de Tienda y Ventas (Vendedor)

- **RF50 - Resumen Financiero de Tienda:** El comerciante consultará ingresos brutos, comisión retenida del 10%, balance líquido y pedidos pendientes en `GET /api/tienda/resumen`.
- **RF51 - Publicación de Productos:** El comerciante podrá subir nuevos productos con imagen multipart en `POST /api/productos`.
- **RF52 - Edición y Control de Stock:** El vendedor podrá modificar precio, descuento, stock y descripción en `PUT /api/productos/:id`.
- **RF53 - Actualización de Envíos:** El vendedor podrá actualizar el estado del pedido (En Camino, Entregado) en `PATCH /api/tienda/pedidos/:id/estado`, notificando al comprador.
- **RF54 - Dispersión Bancaria:** Registro de datos de cuenta (Bancolombia, Nequi, Daviplata) para transferencias de liquidación.

3.6 Módulo Social, Calificaciones y Notificaciones

- **RF80 - Notificaciones en Tiempo Real:** Consulta de bandeja en `GET /api/notificaciones` y marcado de lectura en `PATCH /api/notificaciones/marcar-leidas`.
- **RF84 / RF103 - Calificación de Vendedores:** Al recibir un pedido, el comprador podrá registrar puntuación de 1 a 5 estrellas y reseña en `POST /api/tienda/:vendedorId/calificar`.
- **RF85 - Mensajería Directa:** Canal de chat bidireccional entre comprador y vendedor vía `GET/POST /api/chat/mensajes/:conversacionId`.
- **RF86 - Reportes de Contenido:** Envío de reportes de productos o usuarios indebidos a la mesa de ayuda vía `POST /api/reportes`.

3.7 Módulo de Administración de Plataforma

- **RF90 - Métricas Centrales:** Dashboard administrativo con ventas globales, comisiones CommerCity acumuladas y usuarios registrados en `GET /api/admin/metricas`.
- **RF91 - Moderación de Usuarios y Publicaciones:** Suspensión, reactivación o baja de cuentas/productos en `PATCH /api/admin/usuarios/:id/estado`.

4. MATRIZ DE MAPEO: PANTALLAS / MODALES vs ENDPOINTS API

Pantalla / Modal DOM	Endpoint Backend	Verbo	Cabeceras Requeridas	Payload / Query	Respuesta Backend y Efecto en UI
page-login	/api/usuarios/login	POST	Content-Type: application/json	{ email, password }	Retoma { token, usuario: { id, nombre, roles: [...] } }. Guarda JWT, activa roles y navega a home o admin. Registra el

page-registro	/api/usuarios/registro	POST	Content-Type: application/json	{ nombre, email, password, vendedor: bool }	usuario en MySQL, vincula roles en usuario_roles y auto-autentica.
page-recuperar	/api/usuarios/recuperar	POST	Content-Type: application/json	{ email }	Dispara correo de recuperación y notifica por toast.
page-home	/api/productos	GET	—	? cat=...&q=...&page=1	Carga lista de productos en #home-product-grid y carruseles.
Modal #pd-modal	/api/productos/:id	GET	—	Parámetro URL: id	Carga datos completos de producto y vendedor en modal de detalle.
page-carrito	/api/carrito	GET	Authorization: Bearer <token>	—	Renderiza la lista #carrito-items con subtotales por ítem.
Modal #pd-modal	/api/carrito	POST	Authorization: Bearer <token>	{ producto_id, cantidad }	Agrega ítem, valida stock y actualiza badges #sidebar-cart-badge y #bnav-cart-dot.
cartQty en Carrito	/api/carrito/:id	PATCH	Authorization: Bearer <token>	{ cantidad }	Actualiza cantidad en servidor y refresca resumen.
cartRemove en Carrito	/api/carrito/:id	DELETE	Authorization: Bearer <token>	—	Elimina ítem del carrito en BD.
Modal #pasarela-modal	/api/pedidos/resumen	GET	Authorization: Bearer <token>	—	Recibe { subtotal, iva, comision, total } calculados por backend. Renderiza #pas-subtotal, #pas-iva, #pas-amount.
Proceso Pago (pasarela)	/api/pedidos/confirmar-pago	POST	Authorization: Bearer <token>	{ metodo, datos_pago, direccion_id }	Descuenta stock en BD, genera pedido, vacía carrito y retorna comprobante exitoso.
page-historial	/api/historial/compras	GET	Authorization: Bearer <token>	?estado=...	Renderiza historial de pedidos con estados actualizados.
Cancelar en			Authorization:		Cambia estado a cancelado y

Historial	/api/historial/cancelar/:id	POST	Bearer <token>	{ motivo }	restituye inventario.
page-tienda	/api/tienda/resumen	GET	Authorization: Bearer <token>	—	Muestra ventas, ingresos, comisión (10%) y balance neto en panel vendedor.
page-pedidos	/api/tienda/pedidos	GET	Authorization: Bearer <token>	?estado=...	Lista pedidos de clientes para despacho.
Modal #od-modal	/api/tienda/pedidos/:id/estado	PATCH	Authorization: Bearer <token>	{ nuevo_estado }	Cambia estado a 'En Camino' o 'Entregado' y alerta al comprador.
Modal #add-prod-modal	/api/productos	POST	Authorization: Bearer <token>	multipart/form-data	Guarda nuevo producto en BD con foto en CDN y lo muestra en tienda.
Modal #edit-prod-modal	/api/productos/:id	PUT	Authorization: Bearer <token>	multipart/form-data	Actualiza datos de producto e impacta catálogo.
Panel #notifs-panel	/api/notificaciones	GET	Authorization: Bearer <token>	—	Lista notificaciones del usuario en #notifs-panel.
Limpiar Notificaciones	/api/notificaciones/marcar-leidas	PATCH	Authorization: Bearer <token>	{ ids: [...] }	Marca notificaciones como leidas en BD y apaga badge visual.
page-chat	/api/chat/mensajes/:convId	GET / POST	Authorization: Bearer <token>	POST: { mensaje }	Carga mensajes y envía nuevos mensajes en tiempo real.
Modal #rating-modal	/api/tienda/:vendedorId/calificar	POST	Authorization: Bearer <token>	{ puntuacion, comentario }	Almacena calificación de compra en base de datos.
Modal #report-modal	/api/reportes	POST	Authorization: Bearer <token>	{ entidad_tipo, entidad_id, motivo }	Genera ticket para resolución administrativa.
page-admin	/api/admin/metricas	GET	Authorization: Bearer <token> (Admin)	—	Muestra dashboard con ingresos globales y métricas de ciudad.
Moderación Admin	/api/admin/usuarios/:id/estado	PATCH	Authorization: Bearer <token> (Admin)	{ accion: 'suspender' 'activar' }	Actualiza estado de usuario de MySQL

5. REQUERIMIENTOS NO FUNCIONALES (RNF)

- **RNF01 - Seguridad:** Transmisión cifrada mediante HTTPS/TLS 1.3. Criptografía de contraseñas con bcrypt (factor de costo 10+) en backend. Tokens JWT con algoritmo HMAC-SHA256 y expiración forzada de 24 horas.
- **RNF02 - Rendimiento:** Tiempo de respuesta de endpoints de catálogo inferior a 350 ms bajo cargas estándar. Compresión de imágenes multipart antes del envío.
- **RNF03 - Disponibilidad:** Arquitectura preparada para tolerancia a fallos en BD MySQL con pool de conexiones gestionado

(mysql2 pool con límite de 30 conexiones concurrentes).

- **RNF04 - Usabilidad y Compatibilidad:** Soporte fluido para Android 8.0+ y iOS 13+ empaquetado bajo Ionic Capacitor. Diseño responsive adaptativo (Mobile viewport <= 860px y Desktop).
- **RNF05 - Integridad de Datos:** Aislamiento de transacciones financieras en MySQL con nivel `READ COMMITTED` para evitar compras sobre inventario agotado (*race conditions*).

6. PARÁMETROS TÉCNICOS Y BASE DE DATOS

```
# Configuración del Entorno de Red y Base de Datos Central
API_BASE_URL = "http://localhost:3000"
DB_ENGINE     = "MySQL 8.0"
DB_HOST       = "149.130.178.228"
DB_PORT       = 3306
DB_USER       = "commercity_user"
DB_PASS       = "Thedbcommercity"
DB_NAME       = "commercity_v2"
```

6.1 Esquema Relacional Principal (MySQL `commercity_v2`)

- usuarios: id, nombre, email, password_hash, telefono, direccion, estado, created_at.
- roles: id, nombre (COMPRADOR, VENDEDOR, ADMINISTRADOR).
- usuario_roles: usuario_id, rol_id (Clave foránea compuesta, soporte multirol).
- productos: id, vendedor_id, nombre, descripcion, precio, descuento, stock, categoria, imagen_url, estado.
- pedidos: id, comprador_id, subtotal, iva, comision_plataforma, total, estado, direccion_envio, created_at.
- pedidos_detalle: id, pedido_id, producto_id, cantidad, precio_unitario, subtotal.
- notificaciones: id, usuario_id, titulo, mensaje, leida, tipo, created_at.
- calificaciones: id, vendedor_id, comprador_id, puntuacion, comentario, created_at.

7. CRITERIOS DE ACEPTACIÓN PARA CIERRE DE SPRINT 2

1. **Alineación con Casos de Uso:** El documento SRS cubre el 100% de los identificadores de pantallas y modales para que Yepes vincule los diagramas de secuencia y casos de uso.
2. **Cero Cálculos Financieros en Cliente:** La pasarela de pago delega el 100% del cálculo de Subtotal, IVA (19%) y Comisión (10%) al endpoint `/api/pedidos/resumen`.
3. **Control RBAC:** La interfaz móvil responde exclusivamente a los roles autenticados devueltos por la base de datos `commercity_v2`.
4. **Verificación de Conectividad:** Backend validado en `http://localhost:3000` conectado a la base de datos `commercity_v2` en `149.130.178.228:3306`.